

Value Change Listener en JSF



Value Change Listener en JSF

1. Concepto Básico

Concepto de Value Change Listener

Un **Value Change Listener** en JSF es un mecanismo que permite escuchar y responder a los cambios en el valor de un componente de entrada. Cuando un usuario modifica el valor de un componente y envía el formulario, el listener puede interceptar el cambio y ejecutar una lógica específica antes de que se procese el resto del ciclo de vida de JSF.

Este concepto es útil para validar, modificar datos relacionados o activar procesos basados en el nuevo valor ingresado.

Sintaxis Básica

Configuración en el archivo XHTML

Un componente de entrada que utiliza un **Value Change Listener** se declara utilizando el atributo `valueChangeListener`. Por ejemplo:

```
<h:inputText id="codigoPostal" value="#{candidato.codigoPostal}"
```

```
valueChangeListener="#{vacanteForm.codigoPostallistener}"
immediate="true" />
```

Atributos importantes:

- **valueChangeListener**: Apunta al método del backing bean que manejará el cambio de valor.
- **immediate="true"**: Indica que el listener se debe ejecutar inmediatamente, antes de que se procesen las fases posteriores del ciclo de vida de JSF.

Implementación en el Backing Bean

El método asociado al listener debe aceptar un parámetro de tipo `ValueChangeEvent`. Este objeto contiene información sobre el componente que disparó el evento y los valores antiguo y nuevo.

```
import jakarta.faces.component.UIInput;
import jakarta.faces.component.UIViewRoot;
import jakarta.faces.context.FacesContext;
import jakarta.faces.event.ValueChangeEvent;

public void codigoPostallistener(ValueChangeEvent event) {
    // Obtener el nuevo valor ingresado por el usuario
    String nuevoCodigoPostal = (String) event.getNewValue();

    // Lógica para actualizar otros campos basados en el nuevo valor
    if ("03810".equals(nuevoCodigoPostal)) {
        FacesContext facesContext = FacesContext.getCurrentInstance();
        UIViewRoot uiViewRoot = facesContext.getViewRoot();

        UIInput coloniaInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:colonia");
        coloniaInputText.setValue("Napoles");
        coloniaInputText.setSubmittedValue("Napoles");

        UIInput ciudadInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:ciudad");
        ciudadInputText.setValue("Ciudad de Mexico");
        ciudadInputText.setSubmittedValue("Ciudad de Mexico");

        facesContext.renderResponse(); // Detener el ciclo de vida y enviar la respuesta
    }
}
```

Explicación de Código

1. `event.getNewValue()`: Obtiene el nuevo valor ingresado por el usuario.
2. **Lógica condicional**: Si el código postal es 03810, se actualizan otros campos relacionados como colonia y ciudad.
3. `renderResponse()`: Detiene el ciclo de vida de JSF y renderiza la vista actualizada.

Veamos el proyecto completo aplicando el concepto de Value Change Listener.

Creación del Proyecto Completo

Orden de Creación de Recursos

1. Configuración del proyecto.
2. Creación de las clases del modelo y backing bean.
3. Configuración de las vistas (XHTML).
4. Configuración de internacionalización (archivos de mensajes).
5. Despliegue y prueba en GlassFish.

Paso 1: Configuración del Proyecto

1. **Crea un nuevo proyecto en NetBeans:**
 - Selecciona "Java with Maven" > "Jakarta EE Application".
 - Asigna un nombre al proyecto (por ejemplo, `ValueChangeListener`).
 - Configura el servidor GlassFish como servidor predeterminado.
 - Selecciona la versión "Jakarta EE".
2. **Estructura del Proyecto:** Asegúrate de que la estructura del proyecto tenga los directorios:
 - `src/main/java` para código fuente.
 - `src/main/resources` para archivos de propiedades.
 - `src/main/webapp` para vistas.

Paso 2: Creación de Clases

Clase `Candidato.java`

Descripción: Representa los datos del usuario que se ingresan a través del formulario.

```
package beans.model;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.Date;
```

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("candidato")
@SessionScoped
public class Candidato implements Serializable {

    private Logger log = LoggerFactory.getLogger(Candidato.class);

    private String nombre;
    private String apellido;
    private int salarioDeseado;
    private Date fechaNacimiento;
    private String codigoPostal;
    private String colonia;
    private String ciudad;

    public Candidato(){
        log.info("Creando el objeto Candidato");
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
        log.info("Modificando la propiedad de nombre:" + this.nombre);
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
        log.info("Modificando la propiedad de apellido: " + this.apellido);
    }

    public int getSalarioDeseado() {
        return salarioDeseado;
    }

    public void setSalarioDeseado(int salarioDeseado) {
        this.salarioDeseado = salarioDeseado;
    }
}
```

```
        log.info("Modificando la propiedad de sueldoDeseado: " + this.salarioDeseado);
    }

    public Date getFechaNacimiento() {
        return fechaNacimiento;
    }

    public void setFechaNacimiento(Date fechaNacimiento) {
        this.fechaNacimiento = fechaNacimiento;
    }

    public String getCodigoPostal() {
        return codigoPostal;
    }

    public void setCodigoPostal(String codigoPostal) {
        this.codigoPostal = codigoPostal;
    }

    public String getColonia() {
        return colonia;
    }

    public void setColonia(String colonia) {
        this.colonia = colonia;
    }

    public String getCiudad() {
        return ciudad;
    }

    public void setCiudad(String ciudad) {
        this.ciudad = ciudad;
    }
}
```

Clase VacanteForm.java

Descripción: Backing bean que gestiona la lógica del formulario, como enviar datos y manejar eventos.

```
package beans.backing;
```

```
import beans.model.Candidato;
import jakarta.enterprise.context.RequestScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.component.UIInput;
import jakarta.faces.component.UIViewRoot;
import jakarta.faces.context.FacesContext;
import jakarta.faces.event.ValueChangeEvent;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named
@RequestScoped
public class VacanteForm {

    private Logger log = LoggerFactory.getLogger(VacanteForm.class);

    @Inject
    private Candidato candidato;

    public VacanteForm(){
        log.info("Creando el objeto VacanteForm");
    }

    public void setCandidato(Candidato candidato){
        this.candidato = candidato;
    }

    public String enviar() {
        if ("Juan".equals(candidato.getNombre()) &&
            "Perez".equals(candidato.getApellido())) {
            String msg = "Gracias Juan Perez, te has registrado correctamente.";
            FacesMessage facesMessage =
                new FacesMessage(FacesMessage.SEVERITY_ERROR, msg, null);
            FacesContext.getCurrentInstance().addMessage(null, facesMessage);
            log.info("Usuario registrado correctamente");
            return "index"; // Redirige a la misma página
        }
        log.info("Datos de usuario inválidos");
        return "fallo"; // Redirige a una página de éxito
    }

    public void codigoPostalListener(ValueChangeEvent valueChangeEvent){
        FacesContext facesContext = FacesContext.getCurrentInstance();
    }
}
```

```

UIViewRoot uiViewRoot = facesContext.getViewRoot();
String nuevoCodigoPostal = (String) valueChangeEvent.getNewValue();
if("03810".equals(nuevoCodigoPostal)){
    UIInput coloniaInputText =
        (UIInput) uiViewRoot.findComponent("vacanteForm:colonia");
    String nuevaColonia = "Napoles";
    coloniaInputText.setValue(nuevaColonia);
    coloniaInputText.setSubmittedValue(nuevaColonia);

    UIInput ciudadInputText =
        (UIInput) uiViewRoot.findComponent("vacanteForm:ciudad");
    String nuevaCiudad = "Ciudad de Mexico";
    ciudadInputText.setValue(nuevaCiudad);
    ciudadInputText.setSubmittedValue(nuevaCiudad);

    facesContext.renderResponse();
}
}
}

```

Paso 3: Configuración de las Vistas

Archivo index.xhtml: Ubícalo en `src/main/webapp` y utiliza el código proporcionado.

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
<h:head>
    <title>Bolsa de Trabajo V5</title>
</h:head>
<h:body>
    <h:form id="vacanteForm">
        <h:messages globalOnly="true" layout="table" styleClass="global-messages" />

        <table>
            <tr>
                <td><h:outputLabel for="nombre"
                    value="#{msgs['vacanteForm.nombre']}" /></td>
                <td><h:inputText id="nombre" required="true"
                    value="#{candidato.nombre}" /></td>
                <td><h:message for="nombre" styleClass="error-message" /></td>
            </tr>
            <tr>

```

```

<td><h:outputLabel for="apellido"
    value="#{msgs['vacanteForm.apellido']}" /></td>
<td><h:inputText id="apellido" required="true"
    value="#{candidato.apellido}" /></td>
<td><h:message for="apellido" styleClass="error-message" /></td>
</tr>
<tr>
<td><h:outputLabel for="salarioDeseado"
    value="#{msgs['vacanteForm.salarioDeseado']}" /></td>
<td><h:inputText id="salarioDeseado" required="true"
    value="#{candidato.salarioDeseado}">
    <f:validateLongRange minimum="500" maximum="5000" />
</h:inputText></td>
<td><h:message for="salarioDeseado"
    styleClass="error-message" /></td>
</tr>
<tr>
<td><h:outputLabel for="fechaNacimiento"
    value="#{msgs['vacanteForm.fechaNacimiento']}" /></td>
<td><h:inputText id="fechaNacimiento" required="true"
    value="#{candidato.fechaNacimiento}">
    <f:convertDateTime pattern="dd/MM/yyyy" />
</h:inputText></td>
<td><h:message for="fechaNacimiento"
    styleClass="error-message" /></td>
</tr>
<tr>
<td><h:outputLabel for="codigoPostal"
    value="#{msgs['vacanteForm.codigoPostal']}" /></td>
<td>
<h:inputText id="codigoPostal" required="true"
    value="#{candidato.codigoPostal}" immediate="true"
    valueChangeListener="#{vacanteForm.codigoPostalListener}"
    onchange="this.form.submit()" />
</td>
<td><h:message for="codigoPostal" styleClass="error-message" /></td>
</tr>
<tr>
<td><h:outputLabel for="colonia"
    value="#{msgs['vacanteForm.colonia']}" /></td>
<td><h:inputText id="colonia" required="true"
    value="#{candidato.colonia}" /></td>
<td><h:message for="colonia" styleClass="error-message" /></td>
</tr>
<tr>

```



```

        <td><h:outputLabel for="ciudad"
            value="#{msgs['vacanteForm.ciudad']}" /></td>
        <td><h:inputText id="ciudad" required="true"
            value="#{candidato.ciudad}" /></td>
        <td><h:message for="ciudad" styleClass="error-message" /></td>
    </tr>
</table>
<h:commandButton action="#{vacanteForm.enviar}"
    value="#{msgs['vacanteForm.enviar']}" />
</h:form>
</h:body>
</html>

```

Paso 4: Internacionalización

Archivos de propiedades:

1. mensajes_es.properties (ubicado en: src/main/resources)

```

vacanteForm.nombre=Nombre
vacanteForm.apellido=Apellido
vacanteForm.salarioDeseado=Salario Deseado
vacanteForm.fechaNacimiento=Fecha de Nacimiento
vacanteForm.codigoPostal=Codigo Postal
vacanteForm.colonia=Colonia
vacanteForm.ciudad=Ciudad
vacanteForm.enviar=Enviar

```

1. mensajes_en.properties (ubicado en: src/main/resources)

```

vacanteForm.nombre=Name
vacanteForm.apellido=Last Name
vacanteForm.salarioDeseado=Desired Salary
vacanteForm.fechaNacimiento=Birthdate
vacanteForm.codigoPostal=Zip Code
vacanteForm.colonia=Suburb
vacanteForm.ciudad=City
vacanteForm.enviar=Submit

```

Conclusión

El **Value Change Listener** en JSF es una poderosa herramienta para manejar eventos basados en cambios de valores en los componentes de entrada. Su capacidad de actuar antes de que el modelo sea actualizado permite implementar lógica reactiva de manera eficiente y flexible en aplicaciones web con JSF.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)